



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Práctica No. 4: Creación de clases e instancias de objetos usando constructores.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Itzel Barriba Cázares

27/02/2026

Tabla de contenido

I. Objetivo	2
II. Descripción de la practica.....	3
Lista de materiales.....	3
III. Desarrollo de la práctica	3
Programa Main	3
Programa 1 — Clase Lavadora.....	5
Programa 2 — Clase Factura.....	9
Programa 4 — Clase Empleado.....	11
IV. Análisis de Resultados.....	13
Programa 1 — Lavadora.....	13
Programa 2 — Factura.....	13
Programa 4 — Empleado.....	14
V. Conclusiones	15
VI. Referencias.....	15
VII. Conclusiones	15
VIII. Bibliografía.....	15
Entrega.....	16
Criterios para evaluar:	16
Código (70 puntos)	16
Reporte (30 puntos):	16

I. Objetivo

El alumno diseñará clases para modelar el estado y comportamiento de entidades del mundo real, a través de atributos y métodos, respetando los principios de la orientación a objetos y las capacidades del lenguaje Java. En esta práctica se implementaron tres programas independientes:

- Programa 1: clase Lavadora con atributos asignados mediante setters, sin constructor.
- Programa 2: clase Factura con constructor completo y método para calcular el monto total.
- Programa 4: clase Empleado con constructor, cálculo de salario anual y método para aplicar un aumento porcentual.

II. Descripción de la practica

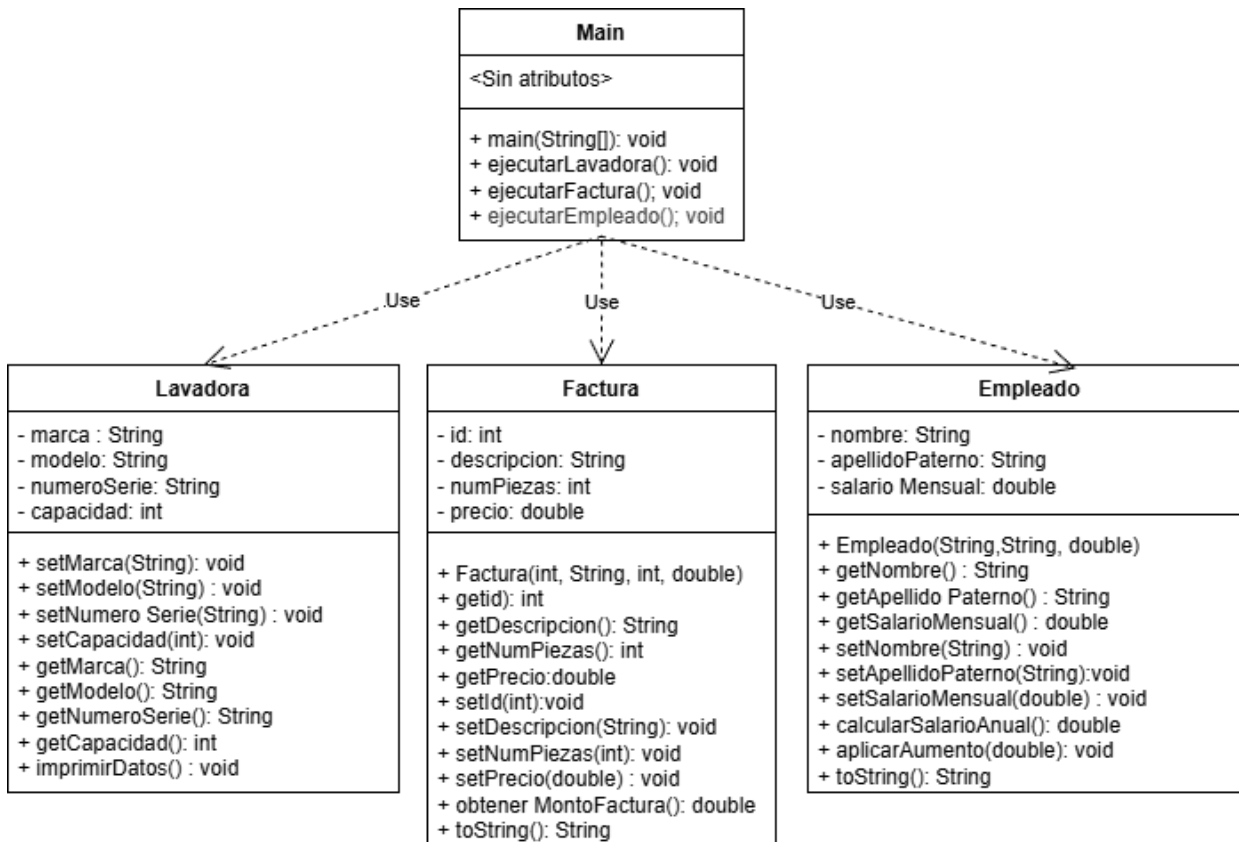
Lista de materiales

- Computadora Personal (PC)
- Ambiente de desarrollo. (Vscode)
- JDK Java Development Kit

III. Desarrollo de la práctica

Diagrama UML

El diagrama muestra las tres clases modelo y la clase Main con sus métodos de prueba. Cada clase de prueba depende de su clase modelo mediante una relación «uses» representada con flecha punteada.



Programa Main

La clase Main actúa como punto de entrada único del programa. Su método main implementa un ciclo do-while que muestra el menú de opciones repetidamente hasta que el usuario ingrese 0. Dentro del ciclo, un switch evalúa la opción seleccionada y llama a la función estática correspondiente.

El método ejecutarLavadora() crea un objeto Lavadora con new Lavadora() sin argumentos, ya que esta clase no tiene constructor parametrizado. Los atributos se asignan uno a uno mediante los setters antes de llamar a lavadora1.imprimirDatos(), que imprime los valores directamente en consola. Esto demuestra que es posible inicializar un objeto sin constructor parametrizado, asignando sus atributos a través de los métodos setter.

El método `ejecutarFactura()` crea dos objetos `Factura` pasando los cuatro atributos directamente al constructor. Para mostrar la información se utiliza `System.out.printf("%s", facturaX.toString())`, que llama explícitamente al método `toString()` definido en la clase `Factura` y pasa el resultado como argumento de formato a `printf`. Esto hace visible en el código que la representación del objeto proviene del método `toString()`.

El método `ejecutarEmpleado()` crea dos objetos `Empleado` usando el constructor, pasando nombre, apellido paterno y salario mensual como argumentos. Primero imprime el salario anual de cada empleado con `System.out.printf("%s", empleadoX.toString())`. Luego llama a `aplicarAumento(10)` en ambos objetos, que modifica internamente el atributo `salarioMensual` incrementándolo en un 10%. Finalmente vuelve a imprimir los datos para mostrar el nuevo salario anual, demostrando que el estado del objeto fue actualizado correctamente tras el aumento.

Programa 1 — Clase Lavadora

La clase `Lavadora` modela una lavadora doméstica con cuatro atributos privados: `marca` y `modelo` (`String`), `numeroSerie` (`String`) y `capacidad` (`int`). Esta clase no tiene constructor definido, por lo que Java asigna automáticamente el constructor por defecto que inicializa todos los atributos con sus valores predeterminados (`null` para `String`, `0` para `int`).

Para asignar valores se utilizan cuatro setters (`setMarca`, `setModelo`, `setNumeroSerie`, `setCapacidad`) y para consultarlos se cuenta con sus correspondientes getters. El método `imprimirDatos()` muestra en consola todos los atributos con etiquetas descriptivas.

En `Main.java` se crea un objeto `Lavadora` con `new Lavadora()` (sin argumentos) y se asignan los valores uno a uno mediante los setters antes de llamar a `imprimirDatos()`. Este programa demuestra que es posible inicializar un objeto sin constructor parametrizado, asignando los atributos a través de sus métodos `setter`.

```

/*-----
Practica 4: Creación de clases e instancias de objetos usando constructores.
Nombre: Joshua Osorio
Materia: Programación Orientada a Objetos
Fecha: Marzo/2026
-----*/
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int opcion;
        do {
            System.out.flush();
            System.out.printf("\n\n\tPráctica 4 - POO\n");
            System.out.println("\n\t----- Menú -----");
            System.out.println("\t1 - Programa 1: Lavadora");
            System.out.println("\t2 - Programa 2: Factura");
            System.out.println("\t3 - Programa 4: Empleado");
            System.out.println("\t0 - Salir");
            System.out.printf("\tOpción:\t");
            opcion = input.nextInt();

            switch (opcion) {
                case 1:
                    ejecutarLavadora();
                    break;
                case 2:
                    ejecutarFactura();
                    break;
                case 3:
                    ejecutarEmpleado();
                    break;
                case 0:
                    System.out.printf("\n\tSaliendo...\n");
                    break;
                default:
                    System.out.printf("\n\tOpción no válida, intente nuevamente.\n");
                    break;
            }

        } while (opcion != 0);
        input.close();
    }
}

```

```

// -----
// Programa 1: crea un objeto Lavadora y muestra sus datos.
// -----
public static void ejecutarLavadora() {
    System.out.println("\n\n\t--- Programa 1: Lavadora ---");

    // Crear objeto sin constructor, asignar valores con setters.
    Lavadora lavadora1 = new Lavadora();
    lavadora1.setMarca("LG");
    lavadora1.setModelo("T16WH4");
    lavadora1.setNumeroSerie("SN-2024-00123");
    lavadora1.setCapacidad(16);

    System.out.println();
    lavadora1.imprimirDatos();
}

// -----
// Programa 2: crea objetos Factura y muestra el monto.
// -----
public static void ejecutarFactura() {
    System.out.println("\n\n\t--- Programa 2: Factura ---");
    // Crear facturas usando el constructor completo.
    Factura factura1 = new Factura(1, "Martillo", 3, 85.50);
    Factura factura2 = new Factura(2, "Tornillos 1/2 pulgada (caja)", 5, 42.00);
    System.out.println();
    System.out.printf("%s", factura1.toString());
    System.out.println();
    System.out.printf("%s", factura2.toString());
}

// -----
// Programa 4: crea dos Empleados, muestra salario anual
// antes y después de un aumento del 10%.
// -----
public static void ejecutarEmpleado() {
    System.out.println("\n\n\t--- Programa 4: Empleado ---");
    // Crear dos empleados con el constructor.
    Empleado empleado1 = new Empleado("Joshua", "Osorio", 12000.00);
    Empleado empleado2 = new Empleado("Maria", "Lopez", 9500.00);
    // Salario anual antes del aumento.
    System.out.println("\n\t=== Salarios ANTES del aumento ===");
    System.out.println(empleado1);
    System.out.println();
    System.out.println(empleado2);
    // Aplicar aumento del 10%.
    empleado1.aplicarAumento(10);
    empleado2.aplicarAumento(10);
    // Salario anual después del aumento.
    System.out.println("\n\t=== Salarios DESPUES del aumento del 10% ===");
    System.out.printf("%s", empleado1.toString());
    System.out.println();
    System.out.printf("%s", empleado2.toString());
}
}

```

```

/*-----
Practica 4: Creación de clases e instancias de
objetos usando constructores.
Nombre: Joshua Osorio
Materia: Programación Orientada a Objetos
Fecha: Marzo/2026
-----*/

```

```

public class Lavadora {

    // Atributos privados de la clase.
    private String marca;
    private String modelo;
    private String numeroSerie;
    private int capacidad;

    // Setters
    public void setMarca(String marca) {
        this.marca = marca;
    }

    public void setModelo(String modelo) {
        this.modelo = modelo;
    }

    public void setNumeroSerie(String
numeroSerie) {
        this.numeroSerie = numeroSerie;
    }

    public void setCapacidad(int capacidad) {
        this.capacidad = capacidad;
    }

    // Getters
    public String getMarca() {
        return marca;
    }

    public String getModelo() {
        return modelo;
    }

    public String getNumeroSerie() {
        return numeroSerie;
    }

    public int getCapacidad() {
        return capacidad;
    }
}

```

```

// Método para imprimir los datos de la
lavadora.
public void imprimirDatos() {
    System.out.println("--- Datos de la
Lavadora ---");
    System.out.println("Marca:\t\t" +
marca);
    System.out.println("Modelo:\t\t" +
modelo);
    System.out.println("Número de serie:\t"
+ numeroSerie);
    System.out.println("Capacidad:\t" +
capacidad + " kg");
}
}

```

Programa 2 — Clase Factura

La clase Factura representa un artículo de ferretería con cuatro atributos privados: id (int), descripcion (String), numPiezas (int) y precio (double). A diferencia de Lavadora, esta clase define un constructor completo que recibe los cuatro valores como parámetros y los asigna a sus respectivos atributos usando this.

El método obtenerMontoFactura() calcula y devuelve el monto total multiplicando precio por numPiezas. El método toString() construye y devuelve un String con todos los atributos formateados, incluyendo el monto total calculado. Se usa String.format("%.2f", valor) para mostrar los valores monetarios con exactamente dos decimales.

En Main.java se crean dos objetos Factura usando el constructor, pasando los datos directamente como argumentos. Se llama a System.out.println(facturaX) que internamente invoca toString() para imprimir la información de cada factura.

```
/*-----  
Practica 4: Creación de clases e instancias de objetos usando constructores.  
Nombre: Joshua Osorio  
Materia: Programación Orientada a Objetos  
Fecha: Marzo/2026  
-----*/  
  
public class Factura {  
  
    // Atributos privados de la clase.  
    private int id;  
    private String descripcion;  
    private int numPiezas;  
    private double precio;  
  
    // Constructor que inicializa las cuatro variables de instancia.  
    public Factura(int id, String descripcion, int numPiezas, double precio) {  
        this.id = id;  
        this.descripcion = descripcion;  
        this.numPiezas = numPiezas;  
        this.precio = precio;  
    }  
  
    // Getters  
    public int getId() {  
        return id;  
    }  
  
    public String getDescripcion() {  
        return descripcion;  
    }  
  
    public int getNumPiezas() {  
        return numPiezas;  
    }  
  
    public double getPrecio() {  
        return precio;  
    }  
  
    // Setters  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public void setDescripcion(String descripcion) {  
        this.descripcion = descripcion;  
    }  
  
    public void setNumPiezas(int numPiezas) {  
        this.numPiezas = numPiezas;  
    }  
}
```

```

public void setPrecio(double precio) {
    this.precio = precio;
}

// Calcula y devuelve el monto total de la factura.
public double obtenerMontoFactura() {
    return precio * numPiezas;
}

// Método toString para imprimir los datos de la factura.
public String toString() {
    return "--- Datos de la Factura ---"
        + "\nId:\t\t" + id
        + "\nDescripción:\t" + descripcion
        + "\nNúmero de piezas:\t" + numPiezas
        + "\nPrecio unitario:\t$" + String.format("%.2f", precio)
        + "\nMonto total:\t\t$" + String.format("%.2f", obtenerMontoFactura());
}
}

```

Programa 4 — Clase Empleado

La clase Empleado modela a un trabajador con tres atributos privados: nombre (String), apellidoPaterno (String) y salarioMensual (double). Se usa double para el salario en lugar de int para soportar cantidades con centavos.

El constructor inicializa las tres variables en un solo paso. El método calcularSalarioAnual() multiplica salarioMensual por 12 y devuelve el resultado como double. El método aplicarAumento(double porcentaje) modifica directamente el atributo salarioMensual sumándole el porcentaje correspondiente: $\text{salarioMensual} += \text{salarioMensual} * (\text{porcentaje} / 100)$. Esto actualiza el estado del objeto para que el nuevo salario quede reflejado en cálculos posteriores.

En Main.java se crean dos objetos Empleado con salarios distintos. Se imprime el salario anual de cada uno antes del aumento, se llama a aplicarAumento(10) en ambos objetos, y se vuelve a imprimir el salario anual después del aumento para verificar el cambio.

```
/*-----  
Practica 4: Creación de clases e instancias de objetos usando constructores.  
Nombre: Joshua Osorio  
Materia: Programación Orientada a Objetos  
Fecha: Marzo/2026  
-----*/  
  
public class Empleado {  
  
    // Atributos privados de la clase.  
    private String nombre;  
    private String apellidoPaterno;  
    private double salarioMensual;  
  
    // Constructor que inicializa las tres variables de instancia.  
    public Empleado(String nombre, String apellidoPaterno, double salarioMensual) {  
        this.nombre = nombre;  
        this.apellidoPaterno = apellidoPaterno;  
        this.salarioMensual = salarioMensual;  
    }  
  
    // Getters  
    public String getNombre() {  
        return nombre;  
    }  
  
    public String getApellidoPaterno() {  
        return apellidoPaterno;  
    }  
  
    public double getSalarioMensual() {  
        return salarioMensual;  
    }  
  
    // Setters  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void setApellidoPaterno(String apellidoPaterno) {  
        this.apellidoPaterno = apellidoPaterno;  
    }  
  
    public void setSalarioMensual(double salarioMensual) {  
        this.salarioMensual = salarioMensual;  
    }  
  
    // Calcula y devuelve el salario anual (12 meses).  
    public double calcularSalarioAnual() {  
        return salarioMensual * 12;  
    }  
}
```

```
// Aplica un aumento porcentual al salario mensual.
public void aplicarAumento(double porcentaje) {
    // Importante mencionar que recibe el porcentaje 67 % y no 0.67
    salarioMensual += salarioMensual * (porcentaje / 100);
}

// Método toString para imprimir los datos del empleado.
public String toString() {
    return "Empleado: " + nombre + " " + apellidoPaterno
        + "\nSalario mensual:\t$" + String.format("%.2f", salarioMensual)
        + "\nSalario anual:\t\t$" + String.format("%.2f", calcularSalarioAnual());
}
}
```

IV. Análisis de Resultados

Programa 1 — Lavadora

Al ejecutar Main, la salida esperada es:

```
> cd "C:\Users\Okuyt\OneDrive - uabc.edu.mx\8_Semestre\Práctica 4 - P00"
Okuyt ~\OneDrive - uabc.edu.mx\8_Semestre
n > javac -d output -sourcepath src src
Okuyt ~\OneDrive - uabc.edu.mx\8_Semestre
n > java -cp output Main

Práctica 4 - P00
----- Menú -----
1 - Programa 1: Lavadora
2 - Programa 2: Factura
3 - Programa 4: Empleado
0 - Salir

Opción: 1

----- Programa 1: Lavadora -----
----- Datos de la Lavadora -----
Marca:      LG
Modelo:     T16WH4
Número de serie: SN-2024-00123
Capacidad:  16 kg
```

Esto confirma que los setters asignan correctamente los valores a los atributos privados y que el método imprimirDatos() los recupera a través de los atributos directamente. También demuestra que un objeto puede ser creado sin constructor parametrizado y configurado paso a paso con setters.

Programa 2 — Factura

Al ejecutar Main opción 2 con dos facturas (3 martillos a \$85.50 y 5 cajas de tornillos a \$42.00), la salida esperada es:

```

— Programa 2: Factura —

— Datos de la Factura —
Id:      1
Descripción: Martillo
Número de piezas: 3
Precio unitario:  $85.50
Monto total:      $256.50

— Datos de la Factura —
Id:      2
Descripción: Tornillos 1/2 pulgada (caja)
Número de piezas: 5
Precio unitario:  $42.00
Monto total:      $210.00

```

Los montos son correctos: $3 \times \$85.50 = \256.50 y $5 \times \$42.00 = \210.00 . Esto valida que el método `obtenerMontoFactura()` realiza el cálculo correctamente y que `toString()` presenta la información de forma legible.

Programa 4 — Empleado

Con empleado1 con salario mensual de \$12,000 y empleado2 con \$9,500, antes del aumento los salarios anuales son \$144,000 y \$114,000 respectivamente. Tras aplicar el 10%, los nuevos salarios mensuales son \$13,200 y \$10,450, con salarios anuales de \$158,400 y \$125,400. La salida esperada es:

```

— Programa 4: Empleado —

=== Salarios ANTES del aumento ===
Empleado: Joshua Osorio
Salario mensual:  $12000.00
Salario anual:    $144000.00

Empleado: Maria Lopez
Salario mensual:  $9500.00
Salario anual:    $114000.00

=== Salarios DESPUES del aumento del 10% ===
Empleado: Joshua Osorio
Salario mensual:  $13200.00
Salario anual:    $158400.00

Empleado: Maria Lopez
Salario mensual:  $10450.00
Salario anual:    $125400.00

```

Esto confirma que el método aplicarAumento modifica correctamente el estado interno del objeto y que calcularSalarioAnual refleja el nuevo salario.

V. Conclusiones

Constructores vs. setters:

La práctica demostró dos formas de inicializar objetos. En el Programa 1 se vio que, sin constructor parametrizado, los atributos se asignan individualmente mediante setters, lo que da flexibilidad pero requiere más líneas de código. En los Programas 2 y 4, el constructor permite inicializar todos los atributos en una sola instrucción, garantizando que el objeto quede en un estado válido desde su creación.

Encapsulamiento:

Los tres programas aplican encapsulamiento declarando todos los atributos como privados. Esto evita que código externo modifique directamente el estado de los objetos, protegiendo la integridad de los datos. Cualquier acceso o modificación pasa por los métodos públicos definidos en la clase.

Métodos de cálculo:

Los métodos obtenerMontoFactura(), calcularSalarioAnual() y aplicarAumento() demostraron que la lógica de negocio debe residir dentro de la clase que maneja los datos. Esto permite reutilizar el comportamiento sin duplicar código en la clase Main.

Método toString():

La implementación de toString() en Factura y Empleado simplifica la presentación de datos, ya que System.out.printf("%s", objeto.toString()) lo invoca y muestra la información. Esta es una buena práctica en Java para facilitar la depuración y la visualización de objetos.

Separación en métodos de prueba:

Separar la lógica del modelo (Lavadora, Factura, Empleado) de la clase de MAIN (ejecutarLavadora, ejecutarFactura, ejecutarEmpleado) sigue el principio de responsabilidad única. Los métodos de ejecutarClase solo crean objetos e invocan métodos, sin contener lógica propia, lo que hace el código más limpio y fácil de mantener.

VI. Referencias

VII. Conclusiones

VIII. Bibliografía

[1] Oracle Corporation. (2024). Java SE Documentation. Recuperado de <https://docs.oracle.com/javase/>

[2] Deitel, P. J., & Deitel, H. M. (2016). Java: How to Program (10th ed.). Pearson Education.

Entrega

- La entrega se realizará mediante una actividad en BB. Se han de entregar los archivos fuentes desarrollados, más el reporte de la práctica adjuntos adjuntos en un fichero zip cuyo nombre esté formado por el número de práctica y nombre del alumno. Ejemplo Practica4JuanLopez.pdf
- Hacer una clase por cada ejercicio y adjuntarlos
- Entregar un reporte de práctica, formato reporte técnico y electrónico, podrá utilizar este formato y agregar el procedimiento correspondiente), que conceptos fundamentales se aplicaron en esta práctica, junto con resultados y conclusiones.

Criterios para evaluar:

Código (70 puntos)

- Debe incluir comentarios en su programa, sin llegar al uso exagerado de ellos.
- Debe utilizar sangrías en los lugares apropiados.
- Poner como encabezado del programa en comentarios, título, la descripción de la práctica, nombre del alumno que realizo la práctica.
- Funcionamiento correcto del programa

Reporte (30 puntos):

Portada, competencia u objetivo	2.5 puntos
Pegar Código del programa (anexar archivos .py)	2.5 puntos
Explicación completa del código y su funcionamiento	15 puntos
Resultados (análisis de resultados)	5 puntos
Conclusiones:	5 puntos

Nota: No se aceptarán trabajos fuera de la fecha de entrega o que no cumplan con las especificaciones de entrega requeridas. Los trabajos que se detecten copiados serán anulados automáticamente para ambos alumnos.